```
$ docker run -it php:8.4-cli
Interactive shell

php > echo "hello world";
hello world
php >
```

By hooking our shell's I/O devices into the container, we can access the `php` binary that is running inside it and merrily hammer away to our heart's content. Another thing we can do with containers is override the default command, by specifying one of our own after the image name:

```
$ docker run -it php:8.4-cli php -m
[PHP Modules]
Core
ctype
curl
...
```

Here we're replacing `php -a` with `php -m`, the output of which is a list of all the available modules that PHP (inside the image) was built with. We can go even further and get our container to read in a file containing PHP code and execute it for us. This requires a little more CLI-fu, as we'll need to pass options to the docker command to tell it how to mount a host filesystem path into the container's RootFS. Let's create a simple PHP source code file, open up your preferred text editor and enter the following text:

```
<?php
echo "hello world" . PHP_EOL;
```

Note the `<?php` line at the very start of the file, this is a an *opening tag*, the PHP interpreter will treat everything after it as executable code until it reads either the end of the file or a closing `?>` tag. From here on in all PHP code snippets assume you have an opening tag in your own code, and are omitted from the listing. Save this as `hello.php` and run it via the `php:8.4-cli` container:

```
$ docker run -v "$PWD":/usr/src/myapp php:8.4-cli php /usr/src/myapp/
hello.php
hello world
```